Bài 7: Fine-tuning nhẹ (PEFT/LoRA/QLoRA) và NLP ứng dụng

AI Engineer – Module 3: AI ứng dụng

Lecture deck – 2,5 giờ

Mục tiêu Bài 7

- Phân biệt text classification, NER/extraction, sentiment và hỏi-đáp tiếng Việt.
- Chọn giữa rule, model NLP chuyên biệt, LLM và fine-tuning cho từng tác vụ.
- Hiểu **PEFT/LoRA** để fine-tune nhẹ mô hình open-source.
- Hiểu **QLoRA** để fine-tune trên **GPU hạn chế**.
- Áp dụng vào phân tích phản hồi khách hàng và phân loại email/ticket tự động.

Bản đồ 2,5 giờ



Tinh thần chính

Xem văn bản như dữ liệu nghiệp vụ có thể phân loại, trích xuất, chuẩn hóa; fine-tune chỉ khi có lý do rõ.

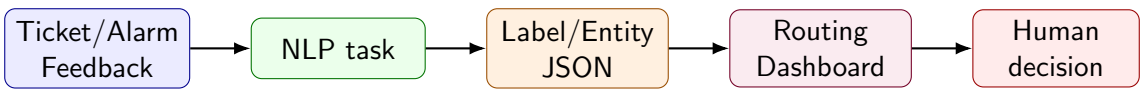
Từ GenAI/RAG sang NLP nghiệp vụ

- Bài 1–3: dùng LLM để sinh, chuẩn hóa và kiểm soát output.
- Bài 5–6: dùng embedding/RAG để hỏi đáp tài liệu có căn cứ.
- Bài 7: quay về dữ liệu text hằng ngày: ticket, alarm, báo cáo, feedback.
- Mục tiêu không chỉ là chatbot, mà là tự động hóa một phần luồng xử lý text.

Case mở đầu: ticket, alarm và phản hồi khách hàng

- Một hệ thống vận hành/CSKH nhận hàng trăm ticket/alarm/feedback mỗi ngày.
- Người điều phối cần biết thuộc nhóm nào, mức nghiêm trọng, entity quan trọng, cảm xúc khách hàng.
- Sai routing làm chậm xử lý incident; bỏ sót alarm có thể gây mất dịch vụ.
- NLP giúp biến text tự do thành dữ liệu có cấu trúc để hỗ trợ quyết định.

Text nghiệp vụ đi vào quy trình như thế nào?



- NLP có giá trị khi output của nó đi được vào một quyết định hoặc hành động cụ thể.
- Nếu không có downstream workflow, model tốt cũng khó tạo giá trị nghiệp vụ.

Bản đồ bài toán NLP

- Text classification: gán nhãn cho ticket, alarm, email, feedback.
- NER / information extraction: lấy trường như site, thời gian, triệu chứng, nguyên nhân nghi ngờ.
- Sentiment / intent: hiểu phản hồi, nhu cầu hoặc mức độ bất mãn.
- QA: trả lời câu hỏi từ tài liệu hoặc đoạn text.
- Summarization và transformation: thường dùng LLM hỗ trợ chuẩn hóa thông tin.

Không phải NLP nào cũng là chatbot

- Chatbot chỉ là một giao diện sử dụng ngôn ngữ tự nhiên.
- Nhiều tác vụ NLP chạy âm thầm trong backend: routing, tagging, extraction, deduplication.
- Đầu ra tốt nhất có thể là nhãn, JSON hoặc flag để người vận hành xem xét.
- Bài học: mô tả task bằng output cần dùng, không bằng tên sản phẩm.

Đầu ra phải gắn với quyết định

- Nhãn ticket dùng để route đội xử lý.
- Entity dùng để điền form, tìm KPI hoặc tra sổ tay tương ứng.
- Severity dùng để ưu tiên queue hoặc cảnh báo supervisor.
- Confidence thấp dùng để đưa vào manual review.
- Mỗi output cần có người dùng và hành động tiếp theo.

Text nghiệp vụ thường rất bẩn

- Viết tắt, lỗi chính tả, tiếng Việt không dấu, xen tiếng Anh/ký hiệu kỹ thuật.
- Nhiều ticket thiếu bối cảnh hoặc gộp nhiều vấn đề trong một câu.
- Alarm có thể ngắn, lặp lại và chứa nhiều mã hệ thống khó đọc.
- Model cần được đánh giá trên dữ liệu giống thực tế, không chỉ ví dụ sạch.

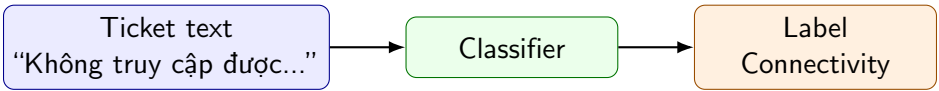
Khung tư duy cho mọi tác vụ NLP

- Input: text nào, độ dài bao nhiêu, có metadata không?
- Task: phân loại, trích xuất, tóm tắt, hỏi đáp hay gợi ý?
- Output: nhãn, entity, JSON, câu trả lời hay action recommendation?
- Evaluation: đúng sai đo bằng gì và lỗi nào nguy hiểm nhất?
- Control: tự động hoàn toàn hay cần con người duyệt?

Ghi nhớ

Đừng chọn model trước khi biết output sẽ được dùng như thế nào.

Phần 1: Text Classification



Định nghĩa ngắn

Text classification là biến một đoạn văn bản thành một hoặc nhiều nhãn đã định nghĩa trước.

Classification: dùng khi nào?

- Routing ticket đến đội xử lý phù hợp.
- Gán mức độ ưu tiên hoặc nhóm sự cố ban đầu.
- Phân loại feedback theo chủ đề.
- Lọc alarm cần xử lý ngay so với alarm nhiễu.
- Theo dõi thống kê xu hướng sự cố theo nhóm.

Một nhãn và nhiều nhãn

- Single-label: mỗi ticket chỉ thuộc một nhóm chính.
- Multi-label: một ticket có thể vừa là connectivity vừa là performance.
- Hierarchical label: nhóm lớn rồi đến nhóm con.
- Output càng phức tạp thì yêu cầu nhãn và evaluation càng chặt.
- Với người mới, nên bắt đầu từ single-label rõ ràng.

Bộ nhãn tốt trông như thế nào?

Nên có

- Tên nhãn rõ nghĩa.
- Định nghĩa ngắn.
- Ví dụ dương/âm.
- Quy tắc khi mơ hồ.

Tránh

- Nhãn chồng lấn.
- Nhãn quá rộng: Other.
- Nhãn hiếm nhưng quan trọng bị bỏ qua.
- Đổi nhãn mà không cập nhật test set.

Ví dụ bộ nhãn ticket

- Connectivity: mất kết nối, chập chờn, timeout, không truy cập được.
- Performance: chậm, packet loss, latency cao, nghẽn băng thông.
- Hardware: lỗi thiết bị, nguồn, module, cổng vật lý.
- Configuration: sai cấu hình, thay đổi rule, VLAN, routing.
- Other/Need review: chưa đủ thông tin hoặc không thuộc các nhóm trên.

Routing ticket là bài toán nghiệp vụ

- Output không chỉ để xem model đúng hay sai.
- Nhãn quyết định đội nào nhận ticket và SLA nào được kích hoạt.
- Sai nhãn có thể gây chuyển vòng, chậm xử lý và tăng chi phí vận hành.
- Vì vậy cần hỏi: nhãn nào sai thì nguy hiểm nhất?

Baseline 1: rule/keyword

- Dùng từ khóa, regex hoặc điều kiện đơn giản.
- Ưu điểm: dễ hiểu, ổn định, rẻ, kiểm soát tốt.
- Nhược điểm: dễ vỡ khi ngôn ngữ thay đổi, khó bao phủ trường hợp mơ hồ.
- Rất nên làm baseline rule trước khi dùng model phức tạp.

Baseline 2: NLP model chuyên biệt

- Dùng model/classifier đã huấn luyện hoặc fine-tune nhỏ cho bộ nhãn.
- Ưu điểm: ổn định hơn LLM trong output nhãn, chi phí inference có thể thấp.
- Nhược điểm: cần dữ liệu nhãn tốt, quy trình train/evaluate rõ ràng.
- Phù hợp khi task lặp lại nhiều và nhãn tương đối ổn định.

Baseline 3: LLM classifier

- Dùng prompt để yêu cầu model chọn nhãn từ danh sách cho trước.
- Ưu điểm: PoC nhanh, xử lý ngôn ngữ đa dạng, giải thích được lý do ở mức hỗ trợ.
- Nhược điểm: có thể không ổn định, cần structured output và validation.
- Phù hợp khi dữ liệu ít, cần thử nhanh hoặc task còn thay đổi.

So sánh nhanh 3 cách phân loại

Tiêu chí	Rule	NLP model	LLM
Khởi động	Rất nhanh	Cần dữ liệu/model	Nhanh cho PoC
Ổn định format	Cao	Cao	Cần ép schema
Linh hoạt ngôn ngữ	Thấp–vừa	Vừa	Cao
Chi phí	Thấp	Thấp–vừa	Thường cao hơn
Giải thích	Dễ audit	Tùy model	Có thể tạo rationale

Class imbalance

- Một số nhãn xuất hiện rất nhiều, nhãn hiếm lại có thể quan trọng.
- Accuracy cao có thể chỉ vì model đoán nhãn phổ biến.
- Ví dụ: 95% ticket bình thường, 5% nghiêm trọng; đoán tất cả là bình thường vẫn đạt 95% accuracy.
- Cần xem precision/recall theo từng lớp, không chỉ accuracy chung.

Confidence không phải sự thật

- Nhiều model trả score hoặc xác suất; LLM có thể tự báo confidence.
- Score cao không luôn đồng nghĩa đúng nếu model chưa được hiệu chỉnh.
- Confidence hữu ích nhất khi dùng để quyết định review threshold.
- Ví dụ: dưới 0.7 thì đưa vào hàng đợi người kiểm tra.

Confusion matrix bằng ngôn ngữ nghiệp vụ

	Dự đoán positive	Dự đoán negative
Thực tế positive	TP bắt đúng	FN bỏ sót
Thực tế negative	FP báo nhầm	TN loại đúng

Câu hỏi chính

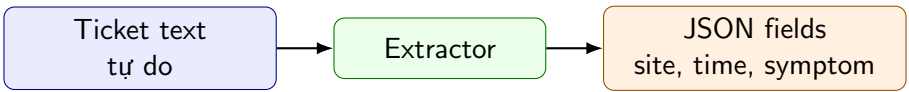
Với use case này, FP hay FN gây hậu quả nặng hơn?

False positive và false negative

- False positive: báo có sự cố hoặc route sai khi thực tế không phải.
- Hậu quả: tăng tải xử lý, tạo false alarm, mất niềm tin vào hệ thống.
- False negative: bỏ sót ticket/sự cố cần xử lý.
- Hậu quả: chậm khắc phục, vi phạm SLA, ảnh hưởng khách hàng.
- Cùng là sai, nhưng chi phí nghiệp vụ khác nhau.

Precision, Recall, F1

- Precision: trong các trường hợp model báo positive, bao nhiêu là đúng?
- Recall: trong các trường hợp thực sự positive, model bắt được bao nhiêu?
- F1: trung bình điều hòa giữa precision và recall.
- Không chọn metric theo thói quen; chọn theo chi phí của lỗi.
- Nếu bỏ sót rất nguy hiểm, recall thường quan trọng hơn precision.



Định nghĩa ngắn

Extraction biến văn bản tự do thành trường dữ liệu có cấu trúc để hệ thống khác sử dụng.

NER khác gì extraction?

- NER thường nhận diện entity như tên người, địa điểm, tổ chức, mã thiết bị.
- Information extraction rộng hơn: lấy trường nghiệp vụ theo schema.
- Trong ticket, ta thường cần site_id, symptom, severity, time, suspected cause, action taken.
- Điểm quan trọng là schema nghiệp vụ, không phải tên kỹ thuật của thuật toán.

Schema extraction mẫu

- `ticket_id`: mã ticket nếu có.
- `site_id`: site hoặc thiết bị liên quan.
- `symptom`: triệu chứng chính.
- `severity`: low / medium / high / critical.
- `time_mentioned`: thời điểm được nhắc đến.
- `suspected_cause`: nguyên nhân nghi ngờ, hoặc unknown.

Ví dụ extraction từ ticket

Input

"Site HN-023 mất kết nối chập chờn từ 09:15, nhiều user báo timeout. Đã reboot router nhưng chưa ổn định."

Output mong muốn

Field	Value
site_id	HN-023
symptom	mất kết nối chập chờn, timeout
time_mentioned	09:15
action_taken	reboot router
suspected_cause	unknown

Slot filling thay vì viết lại tự do

- Khi output dùng cho hệ thống, cần field cố định và kiểu dữ liệu rõ.
- Không để model tự đặt thêm key nếu hệ thống downstream không xử lý được.
- Trường thiếu nên trả `null` hoặc `unknown`, không suy diễn.
- Nên có validation sau extraction.

Rule/regex cho extraction

- Rất tốt cho mã có pattern rõ: site ID, ticket ID, số điện thoại, thời gian.
- Dễ kiểm thử và giải thích.
- Kém linh hoạt với triệu chứng mô tả tự do hoặc ngôn ngữ mơ hồ.
- Thực tế thường kết hợp regex cho field chắc chắn và model cho field mềm.

NER model chuyên biệt

- Phù hợp khi có nhiều ví dụ entity đã gán nhãn.
- Có thể ổn định và rẻ khi triển khai quy mô lớn.
- Cần guideline annotation nhất quán.
- Khó khi schema thay đổi nhanh hoặc entity quá đặc thù chưa có dữ liệu.

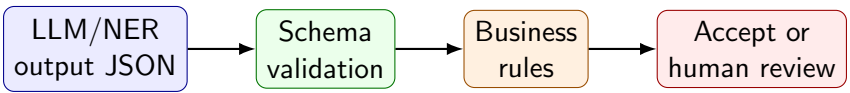
LLM extraction

- Phù hợp để PoC nhanh hoặc schema còn đang điều chỉnh.
- Cần yêu cầu output JSON theo schema, allowed values và quy tắc unknown.
- Cần kiểm tra parse JSON và validate type/value.
- Không để LLM tự bịa entity chỉ vì output cần đủ trường.

Quy tắc unknown rất quan trọng

- Nếu text không nêu nguyên nhân, output phải là unknown.
- Nếu có nhiều khả năng, có thể trả danh sách hypothesis kèm bằng chứng.
- Không biến suy đoán thành fact.
- Trong use case rủi ro cao, unknown tốt hơn câu trả lời tự tin nhưng sai.

Validation sau extraction



- Schema kiểm tra format; business rule kiểm tra logic nghiệp vụ.
- Ví dụ: severity chỉ được nằm trong tập cho phép; site ID phải tồn tại trong danh mục.

Sentiment và intent

- Sentiment: tích cực, tiêu cực, trung tính hoặc mức độ bức xúc.
- Intent: người dùng muốn báo lỗi, hỏi thông tin, yêu cầu thay đổi, khiếu nại.
- Trong vận hành, sentiment ít quan trọng hơn intent/severity ở nhiều case.
- Không nên dùng sentiment chung chung nếu không gắn với hành động tiếp theo.

QA cho văn bản nghiệp vụ (tiếng Việt)

- QA có thể là hỏi đáp trên một đoạn text ngắn hoặc trên kho tài liệu.
- Nếu cần trả lời dựa trên tài liệu nội bộ, Bài 5–6 đã dùng RAG.
- Nếu chỉ cần trích thông tin từ một ticket, extraction có thể phù hợp hơn QA.
- Câu hỏi thiết kế: output là câu trả lời cho người đọc hay dữ liệu cho hệ thống?

Đặc thù tiếng Việt

Có dấu/không dấu, viết tắt, trộn tiếng Anh; đồng nghĩa nghiệp vụ (rớt mạng, mất kết nối, down link, timeout). Phải test trên dữ liệu tiếng Việt thực tế.

Phần 3: Khi nào cần fine-tuning?

- Prompt/few-shot đã tối ưu nhưng hành vi vẫn chưa đủ ổn định ở quy mô.
- Có dữ liệu nhãn/đầu ra chuẩn, task lặp lại nhiều, cùng dạng.
- Prompt quá dài/tốn chi phí, hoặc cần model nhỏ chạy nội bộ ổn định.

Không dùng fine-tuning để

Nhét tri thức thay đổi liên tục (đó là việc của RAG), hay "chữa" dữ liệu kém và nhãn nhiễu.

Full fine-tuning vs PEFT

Full fine-tuning

Cập nhật **toàn bộ** tham số model. Chất lượng cao nhưng tốn rất nhiều GPU, bộ nhớ và thời gian; mỗi task một bản copy model đầy đủ.

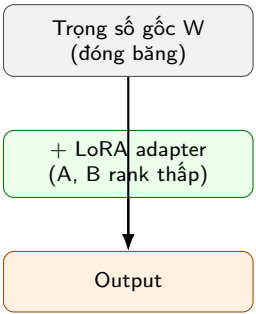
PEFT (Parameter-Efficient Fine-Tuning)

Chỉ huấn luyện **một phần nhỏ** tham số thêm vào, giữ nguyên trọng số gốc. Rẻ hơn nhiều, nhiều task chia sẻ một base model.

LoRA và QLoRA là hai kỹ thuật PEFT phổ biến cho mô hình open-source.

LoRA – Low-Rank Adaptation

- **Đóng băng** trọng số gốc của model.
- Chèn các ma trận rank thấp (adapter) vào một số lớp; chỉ huấn luyện phần này.
- Số tham số cần train giảm hàng trăm lần $\rightarrow$ nhanh, nhẹ, dễ lưu nhiều adapter cho nhiều task.



Khi inference, chỉ cần base model + adapter nhỏ; có thể gộp adapter vào model nếu muốn.

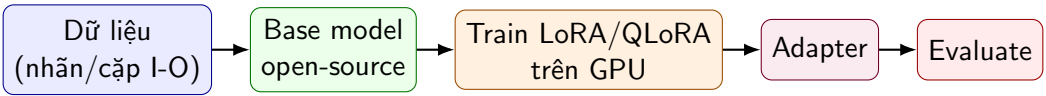
QLoRA – fine-tuning trên GPU hạn chế

- QLoRA = **quantization** (vd: nạp base model ở 4-bit) + **LoRA adapter**.
- Lượng tử hóa base model giảm mạnh bộ nhớ GPU cần thiết; adapter vẫn train ở độ chính xác cao hơn.
- Cho phép fine-tune mô hình vài tỷ tham số trên **một GPU phổ thông**.

Ý nghĩa thực dụng

QLoRA là cách khả thi nhất để tùy chỉnh mô hình open-source khi hạ tầng GPU hạn chế, vẫn giữ chất lượng gần LoRA.

Quy trình fine-tune LoRA/QLoRA



- Chuẩn bị dữ liệu sạch, đúng format (vd: text + label, hoặc instruction + response).
- Chia train/validation/test; giữ test set không dùng để train.
- Chạy huấn luyện trên GPU; theo dõi loss và đánh giá trên test set.

Khung code QLoRA (khái niệm, PyTorch/GPU)

Pseudo-code dùng transformers + peft + bitsandbytes

```
import torch
assert torch.cuda.is_available() # ưu tiên chạy trên GPU
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model

bnb = BitsAndBytesConfig(load_in_4bit=True,
  bnb_4bit_quant_type="nf4", bnb_4bit_compute_dtype=torch.bfloat16)
base = AutoModelForCausalLM.from_pretrained(MODEL, quantization_config=bnb,
  device_map="cuda")
lora = LoraConfig(r=16, lora_alpha=32, target_modules=["q_proj","v_proj"],
  lora_dropout=0.05, task_type="CAUSAL_LM")
model = get_peft_model(base, lora)
model.print_trainable_parameters() # chỉ % nhỏ tham số được train
```

Chi tiết (Trainer/SFTTrainer, dataset, hyperparameters) tùy thư viện; học viên tự chạy.

Đánh giá model fine-tuned

- So sánh với baseline: prompt zero/few-shot, model chưa fine-tune, rule.
- Đo trên test set giữ riêng: precision/recall/F1 theo nhãn, hoặc chất lượng câu trả lời.
- Kiểm tra **overfitting**: tốt trên train nhưng kém trên dữ liệu mới.
- Kiểm tra regression: fine-tune cho task này có làm hỏng khả năng khác không?

Fine-tuning không giải quyết

Privacy, hallucination hay dữ liệu kém – các vấn đề này vẫn cần kiểm soát riêng (Bài 2).

Fine-tuning vs Prompt vs RAG

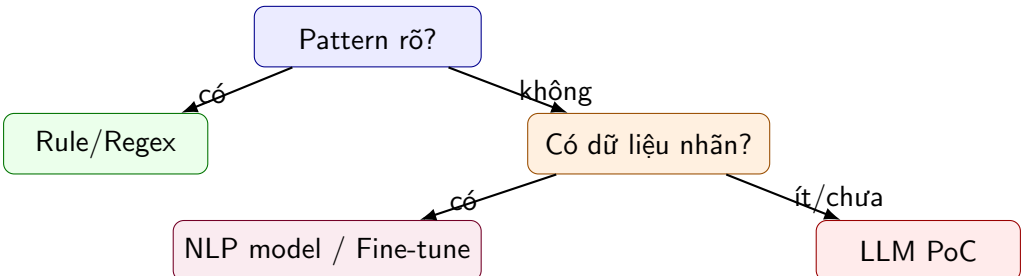
Giải pháp	Dùng khi thiếu gì?	Lưu ý
Prompt	Thiếu chỉ dẫn rõ	Nhanh, rẻ, thử trước
RAG	Thiếu tri thức nội bộ/cập nhật	Cập nhật index, không train lại
Fine-tuning	Thiếu hành vi/format ổn định ở quy mô	Cần dữ liệu tốt; LoRA/QLoRA để rẻ

Không phải thang bậc bắt buộc; chọn theo thiếu hụt thực sự. Fine-tune là bước cân nhắc sau khi prompt/RAG đã tới hạn.

Rule vs NLP model vs LLM vs Fine-tuning

Nhu cầu	Rule	NLP/Fine-tuned	LLM
Pattern rõ	Rất tốt	Không cần thiết	Dư thừa
Nhãn ổn định, volume lớn	Có thể baseline	Rất phù hợp	Cần cân chi phí
Text đa dạng, ít dữ liệu	Khó bao phủ	Khó train	Phù hợp PoC
Output phải parse được	Tốt	Tốt	Cần schema/validator
Giải thích/audit	Dễ	Tùy	Cần logging/prompt

Decision tree đơn giản



Chi phí, độ trễ và privacy

- Rule gần như không có chi phí inference và độ trễ rất thấp.
- Model nhỏ (hoặc fine-tuned) thường phù hợp xử lý batch hoặc real-time volume lớn.
- LLM API linh hoạt nhưng có chi phí, độ trễ và vấn đề dữ liệu nhạy cảm.
- Ticket/feedback có thể chứa thông tin khách hàng; phân loại dữ liệu trước khi gửi vào AI service (Bài 2).

Khi nào không dùng LLM?

- Task chỉ cần keyword/rule rõ ràng và dễ bảo trì.
- Yêu cầu output tuyệt đối ổn định nhưng chưa có validation.
- Dữ liệu quá nhạy cảm và chưa có chính sách sử dụng model phù hợp.
- Chi phí/độ trễ không đáp ứng yêu cầu vận hành.
- Không có cách đo lỗi và không ai chịu trách nhiệm khi model sai.

Dữ liệu nhãn là tài sản

- Muốn đánh giá (và fine-tune) cần tập mẫu có nhãn đáng tin cậy.
- Nhãn phải được định nghĩa bằng guideline, không chỉ cảm tính của một người.
- Nên lưu cả text, nhãn, người gán nhãn, ngày và ghi chú trường hợp mơ hồ.
- Dữ liệu nhỏ nhưng sạch thường hữu ích hơn dữ liệu lớn nhưng nhãn nhiễu.

Annotation guideline

- Mô tả mỗi nhãn bằng định nghĩa ngắn.
- Cho ví dụ đúng và ví dụ dễ nhầm.
- Quy định thứ tự ưu tiên khi ticket có nhiều vấn đề.
- Quy định khi nào dùng Other/Need review.
- Cập nhật guideline khi phát hiện lỗi lặp lại.

Ticket mơ hồ và ngữ cảnh ngoài text

- Một ticket có thể thiếu thông tin, chứa hai sự cố, hoặc dùng thuật ngữ nội bộ.
- Không phải mọi trường hợp mơ hồ đều nên bắt model đoán; có thể cần hỏi lại hoặc review.
- Metadata (site, khách hàng, loại thiết bị, thời điểm, KPI) có thể quyết định nhãn.
- Thiết kế input là một phần của thiết kế giải pháp NLP.

Ứng dụng: phân tích phản hồi khách hàng

- Input: phản hồi/khiếu nại khách hàng tiếng Việt (CSKH, khảo sát, kênh nội bộ).
- Task: sentiment + phân loại chủ đề + trích entity (dịch vụ, khu vực, vấn đề).
- Output: dashboard xu hướng bất mãn theo chủ đề/khu vực; cảnh báo khi tiêu cực tăng.
- Fine-tune nhẹ (LoRA) một model open-source tiếng Việt khi volume lớn và nhãn ổn định.

Giá trị

Phát hiện sớm vấn đề lặp lại, ưu tiên xử lý khách hàng bức xúc, đo chất lượng dịch vụ.

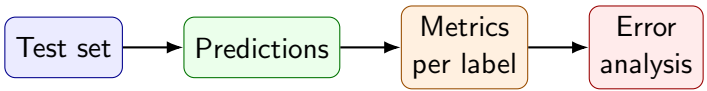
Ứng dụng: phân loại email/ticket tự động

- Input: email/ticket đến hàng ngày với nội dung tự do.
- Task: classification (nhóm/đội xử lý, severity) + extraction (site, thời gian, triệu chứng).
- Output: tự gợi ý routing + điền trường, giảm thời gian phân loại thủ công.
- Model: rule baseline $\rightarrow$ NLP model/fine-tuned classifier cho nhãn ổn định, volume lớn.

An toàn

Nhãn rủi ro cao hoặc confidence thấp chuyển human review; không auto-close ticket.

Đánh giá classification



- Luôn giữ một tập kiểm thử nhỏ (20–50 mẫu), không dùng để viết prompt/rule/train.
- Xem lỗi theo nhãn, theo loại text, theo độ dài và theo mức rủi ro.

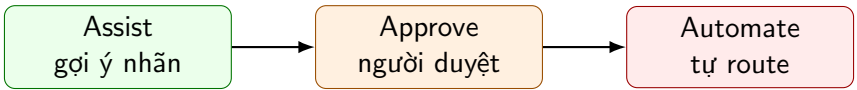
Error taxonomy và error analysis

- Sai do nhãn chồng lấn hoặc guideline không rõ.
- Sai do text thiếu thông tin hoặc chứa nhiều vấn đề.
- Sai do model không hiểu thuật ngữ nội bộ.
- Sai do output không đúng schema hoặc dữ liệu ngoài phân phối.

Không đổ lỗi cho model

Một lỗi có thể chỉ ra nhãn cần sửa, input cần thêm metadata hoặc workflow cần human review. Không phải lỗi nào cũng giải bằng model lớn hơn.

Assist – Approve – Automate cho NLP



- Mức tự động hóa phụ thuộc hậu quả khi sai, độ đo chất lượng và khả năng đảo ngược.
- Bắt đầu bằng Assist thường an toàn hơn; confidence thấp/nhãn rủi ro cao $\rightarrow$ review.

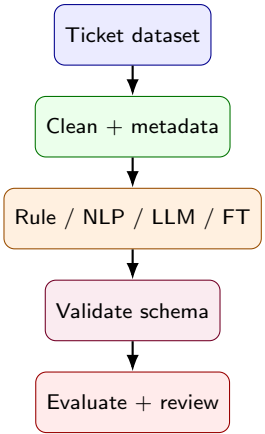
Human-in-the-loop và tích hợp

- Confidence thấp: chuyển người duyệt; nhãn rủi ro cao: xác nhận trước khi route/escalation.
- Entity quan trọng bị thiếu: hỏi lại hoặc gắn flag incomplete.
- Output lưu cùng timestamp, model version, prompt/rule version và confidence; có dashboard xem lỗi.
- Mẫu đã duyệt quay lại làm dữ liệu cải thiện hệ thống và dữ liệu fine-tune.

Checklist và kiến trúc PoC NLP

Checklist

- Bộ nhãn/schema rõ?
- Test set + error analysis?
- So sánh rule/model/LLM/fine-tune?
- Validation + xử lý unknown?
- Ai duyệt lỗi rủi ro cao?



Privacy và bảo mật dữ liệu text

- Ticket/feedback có thể chứa thông tin khách hàng, địa chỉ, số điện thoại hoặc topology nội bộ.
- Cần masking/redaction nếu dùng dữ liệu lớp học hoặc API bên ngoài.
- Logging prompt/output và dữ liệu fine-tune cũng là dữ liệu cần bảo vệ.
- Không dùng dữ liệu thật khi chưa có quyền và chính sách rõ ràng (Bài 2).

Tổng kết Bài 7 và cầu nối

- NLP biến text nghiệp vụ thành nhãn, entity, JSON, insight dùng trong quy trình.
- Chọn rule / NLP model / LLM / fine-tuning theo dữ liệu, chi phí, rủi ro và độ ổn định.
- PEFT/LoRA fine-tune nhẹ mô hình open-source; QLoRA cho phép fine-tune trên GPU hạn chế.
- Ứng dụng: phân tích phản hồi khách hàng, phân loại email/ticket tự động.
- Đánh giá phải đi kèm precision/recall hoặc error table; human review là một phần thiết kế.

Bài 8

Khi tác vụ cần nhiều bước và gọi công cụ bên ngoài: **kiến trúc AI Agent – Planning, Memory, Tool Use**.

Knowledge check và cầu nối sang Lab

- Một ticket có nhãn đúng nhưng thiếu site_id có đủ dùng không?
- Với bài toán bỏ sót sự cố nghiêm trọng, nên ưu tiên precision hay recall?
- Khi nào regex tốt hơn LLM? Khi nào fine-tune đáng làm?
- Vì sao trường unknown cần được cho phép trong extraction?
- Lab sẽ xây PoC Ticket Intelligence: classification + extraction + (mở rộng) fine-tune LoRA/QLoRA + error analysis.